

An open platform

An open platform

Converged is meant to be an open manufacturing platform, not a sealed SaaS box. The core ships under an open-source licence, and integrations, microservices, microfrontends, workflows and applied solutions grow around it.

On the manufacturing market this is practical rather than ideological. Shops differ in equipment, materials, quality standards and supply chains. A closed system stops at the boundary of what its vendor anticipated. An open one lets you add an adapter for one specific machine, or an integration with one specific carrier, without waiting for somebody else's release.

An extension can be small: a machine adapter, an import from an old website, one extra screen, a workflow for an industry-specific process. If it is useful to others, it lands in the shared registry. If it is not, it stays in your deployment and keeps working.

Open does not mean uncontrolled. A module must declare its area of responsibility, respect the architectural boundaries, and never obtain wider data access than it needs. Trust in the ecosystem rests on the sources, on reproducible builds and on an explicit permission model — not on promises.

The module registry

The module registry

The registry is not a separate document and not a database. It is the source tree itself.

```
modules/    |— microservices/<domain>/ms-<name> a data domain and its
API         |— microfrontends/<domain>/mf-<name> a screen mounted at runtime
            |— workflows/wf-<name> a process for the DAG runtime    |—
types/<domain>/ NRPC contracts    |— solutions/ which modules ship
together
```

A module exists because its directory exists. It belongs to a domain because it sits in that domain's folder. It belongs to a solution because

`solutions/solutions.json` names it. There is no fourth place where any of this has to be repeated — which is why the ecosystem page on the site is produced by walking the tree rather than by editing a list.

A module's purpose is taken from its `README.md` : the first paragraph under `## Purpose` (for microfrontends, `## UI Purpose`) and the paragraph under the ownership-boundary heading. Those two paragraphs are the module's contract in plain language, and every module owes them.

A product layer on top of the base — `club` , for instance — is laid out the same way and may drop the domain level: its modules sit directly in

`modules/microservices/ms-<name>` . The build understands both layouts.

Adding a module

Adding a module

The steps are the same for the base platform and for a product layer.

1. **Create the directory** by convention: `modules/microservices/<domain>/ms-<name>` for a service, `modules/microfrontends/<domain>/mf-<name>` for a screen, `modules/workflows/wf-<name>` for a process.
2. **Declare the contract** in `modules/types/<domain>/` and generate the clients with `bun run gen`. The client appears as a `g-<name>` package, usable from the browser, from another process on the bus, and from inside a workflow.
3. **Write the README** with a `## Purpose` section and an ownership-boundary section. The first paragraph of each ends up in the registry on the site — write them for a reader, not for yourself.
4. **Add the module to a solution** if it does not ship alone: put its short name in `modules/solutions/solutions.json` and declare its dependencies.
5. **Rebuild the docs**: `bun run docs` in `core/tools/docs`. The module shows up in the registry and the counters on the ecosystem page recount themselves.

What you do not have to do: edit module lists in the site data, restate the description in the landing, or register the module anywhere else. Generation runs one way — from sources into data, never back. Anything under `data/` is overwritten by the next build.

What review asks of a module: it does not reach into another module's storage, does not bypass the bus with direct calls, declares only the permissions it actually uses, and does not quietly widen its area of responsibility.